

00 DEMO ÖNCESİ — 2 DAKİKA

sunumdan önce yap

```
cd ~/Desktop/Data-Science-Reports/reliability_study/cua_lab
python3 -m cua_lab.cli
```

- Terminali **en az 100 sütun** yap, yazı tipini büyüt.
- Bir kez prova koş** — ilk çalıştırmada Python `__pycache__` yazıyor, o gecikme sunumda olmasın.
- `--model hf` kullanacaksan **interneti kontrol et**; kullanmayacaksan hiç açma.
- Bu ekranı açık bırak: docs/zihniyetler.md — soru gelirse kaynağı gösterirsin.

Anlatım sırası — teori ile demo iç içe

Brief üç teslimat istiyor. Demoyu **ikinci teslimatın kanıtı** olarak değil, **birinci teslimatın anlatım aracı** olarak kullan: her teorik iddiadan sonra onu ekranda göster.

BRIEF MADDESİ	DEMO ADIMI
"kontROLSÜZ koşum öngörülemez tüketir"	1
"kontrollerin devreye girdiğini gösteren PoC"	2
"çalışma prensibi, tespit mantıkları"	3 • 4
"ve sınırları"	5 • 6
"max steps / replans / tokens / süre"	7
"avantajlar, dezavantajlar"	5 • 6

01 PROBLEM — KONTROLSÜZ KOŞUM

~2 dk

```
cua ► --strategy → 17 (none)
cua ► Formu doldur ve gönder
```

```
CEILING · hard_ceiling
adım 300 token 105000 maliyet $1.0500
```

SÖYLE

"Bu **Anthropic'in referans computer-use döngüsü**. Kodun kalbi `while True` — tur sayacı yok, döngü tespiti yok, bütçe yok. Tek çıkış modelin araç çağırmayı bırakması.

300'de durdu çünkü **ben demo güvenliği için bir tavan koydum**. Gerçek referans kodda o tavan da yok."

Brief'in cümlesi: "kontROLSÜZ bir koşum döngüye girip token ve GPU kapasitesini öngörülemez şekilde tüketebiliyor." — İşte bu.

02 KONTROL DEVREDE

~1 dk

```
cua ► --strategy → 5 (openhands-stuck)
cua ► Formu doldur ve gönder
```

```
STUCK · cycle_k2
adım 3 token 1050 maliyet $0.0105
```

durma raporu

```
neden durdu : tekrar deseni tespit edildi (cycle_k2)
ne denendi  : 3 adım, son eylem: left_click({'x':200,'y':200})
ne bulundu  : ekran son 4 adımda 1 farklı durum gordu
sonraki adım : farklı bir yaklaşımla ya da insan müdahalesi
```

SÖYLE

"Aynı model, aynı bozuk ortam. Tek fark bir dedektör: **300 adım → 3. 105 bin token → 1050. Bir dolar → bir sent.**

Ama asıl bakmanızı istediğim **alt kısım**: sadece durmadı, **ne olduğunu da söyledi**. 'cycle_k2' — iki adımlık bir çevrimde dönüyordu. Bu bilgi olmadan sadece 'çok adım attı' bilirdiniz."

100× fark.

03 AYNI PROBLEM, ÜÇ FARKLI TEŞHİS

~3 dk · demonun kalbi

Aynı görevi üç zihniyetle koştur: **Üçü de doğru durduruyor — ama farklı şey söylüyorlar.**

```
cua ► --strategy → 9 (loopguard-dignity)
cua ► Formu doldur ve gönder
cua ► --strategy → 1 (arize-control)
cua ► Formu doldur ve gönder
```

openhands-stuck	STUCK	cycle_k2	3 adım
loopguard-dignity	NEEDS_INPUT	abstain_need_input	7 adım
arize-control	BUDGET_EXHAUSTED	max_steps	13 adım

loopguard'ın raporunu oku

```
neden durdu : ilerleme durdu — devam etmek yerine çekimser
              kalıyorum (yenilgi değil, ayrı bir sonuç)
sonraki adım : SORU: hedef buton tiklanabilir durumda mı?
              Cevap gelmezse elimdekiyle raporlarım.
```

SÖYLE

"Üç farklı **terminal durum**: **STUCK, NEEDS_INPUT, BUDGET_EXHAUSTED**. Üçü de doğru. Ama teşhisleri farklı:

- openhands** deseni adlandırıyor — 'iki adımlık çevrim'
- loopguard** kullanıcıya **soru soruyor** — çekimser kalmak yenilgi değil, ayrı bir sonuç
- arize** sadece 'çok oldu' diyor — ama **kandırılmaz**

Çalışmamın bütün tezi bu farkta: **'ne zaman dur' tek soru değil. 'Durunca ne söyle' de bir soru.**

→ `--info` bas: seçili zihniyetin **kaynağını, neden gerektiğini, tetiklenince ne yaptığını ve neyi kaçırdığını** ekranda gösterir.

04 KÖR NOKTA: A-B-A-B

~1 dk

```
cua ▶ --model → 1 (alternating)
cua ▶ --strategy → 5 (openhands-stuck)
cua ▶ Formu doldur ve gönder
cua ▶ --strategy → 8 (strands-entropy)
cua ▶ Formu doldur ve gönder
```

```
openhands-stuck cycle_k4 5 adım
strands-entropy low_diversity 6 adım
```

SÖYLE

"Model artık **iki nokta arasında gidip geliyor**: A-B-A-B. Bu dizide **hiçbir çağrı bir öncekiyle aynı değil** — ardışık tekrar sayan dedektörlerin sayacı her adımda sıfırlanır ve hiç konuşmaz.

Taradığım **ilk 22 harness'ın sadece 2'si** bunu yakalıyordu.

İkisi de yakaladı ama farklı yoldan: openhands **çevrim tarıyor**, strands **tek satırla çeşitlilik sayıyor** — `len(set(pencere)) < esik.`"

05 YANLIŞ POZİTİF — DEMONUN EN ÖNEMLİ EKRANI

~2 dk

```
cua ▶ --scenario → 3 (flaky · MESRU retry)
cua ▶ --model → 5 (solver)
cua ▶ --compare
```

strateji	durum	sebebi	adım	token	\$
none	OK	finished	6	1900	0.019
openhands-stuck	OK	finished	6	1900	0.019
loopguard-dignity	OK	finished	6	1900	0.019
arize-control	OK	finished	6	1900	0.019
pi-signature	OK	finished	6	1900	0.019
...	16 satirin 16'si da BİREBİR AYNI				

SÖYLE — burada yavaşla

"Ortam hâlâ bozuk: flaky senaryosunda araç iki kez hata veriyor, üçüncüde çalışıyor. Bu **meşru bir retry** — kesilmemesi gerekiyor.

On altı sütunun on altısı da none ile birebir aynı. Aynı adım, aynı token, aynı dolar.

Yani guardrail **çalışan bir koşuma tek token bindirmiyor** ve meşru tekrar denemeyi kesmiyor.

Bir dedektörün ikinci sınavı, yakalamaması gerekeni rahat bırakmaktır. Yalnızca yakaladıklarını gösteren bir demo, yanlış pozitif oranı hakkında hiçbir şey söylemez — 'her koşumu döngü sayan' bir dedektör de o demoyu geçirdi."

108 kombinasyonda 0 yanlış pozitif, ve bunu bir test kilitliyor.

06 SINIRLAR: KÖR NOKTALAR VE ÇAKIŞMA

~2 dk · dezavantaj bölümü

```
cua ▶ --scenario → 2 (dead_button)
cua ▶ --model → 6 (stubborn)
cua ▶ --compare
```

BU KOSUMDA AYIRT EDİLEMEYENLER (1 kume)

```
autogen-static = galileo-breaker = improvement-loop
= none = telemetry-repair = verify-gate = voi-allocation
galileo-breaker: hata ORANINA bakar; broken_tool ile ayırır
verify-gate: bitirme iddiası bekler; --model liar ile ayırır
improvement-loop: MUDAHALE ETMEZ; farkı rapor alanında
autogen-static: hiçbir butce eksenini acik degilken ayırır
```

SÖYLE

"Altı strateji burada **hiç durduramadı** — ve bu bir kusur değil, **kör noktaları**. verify-gate bitirme iddiası bekliyor, ajan hiç 'bitirdim' demiyor; galileo-breaker hata oranına bakıyor, bu senaryoda hiç hata yok.

PoC bunu **kendisi raporluyor** ve her biri için **ayrıştırıcı koşulu** veriyor. İki ayrı satır görmek, iki ayrı zihniyet görmek değildir.

Buradan çıkan ders: tek katman yetmiyor. --strategy 'de virgülle üst üste koyabiliyorsun."

```
cua ▶ --scenario → 1 (healthy) · --model → 3 (liar) · --compare
none OK finished 1 adım ← yapılmamış is'i teslim etti
verify-gate DEGRADED verify_failed 3 adım ← reddetti
```

07 BÜTÇE EKSENLERİ — BRIEF'İN KALEMLERİ

~2 dk

```
cua ▶ --budget
max_steps [12] ▶ 3
cua ▶ --strategy → 5 (openhands-stuck) · görev
cua ▶ --strategy → 1 (arize-control) · görev
```

```
--max-steps 3 openhands-stuck cycle_k2 ← limit ETKİSİZ
--max-steps 3 arize-control max_steps ← limit ETKİLİ
```

SÖYLE

"Brief dört kalem istiyor: **max steps / replans / tokens / süre**. Ben bir de **dolar** eksenini ekledim — token bir vekil ölçü, iki model arasında on beş kat fiyat farkı olabilir.

Ama asıl göstermek istediğim şu: **aynı limit, farklı sonuç**. openhands-stuck bir pencere dedektörü, bütçe kancası hiç yok — limiti umursamıyor. arize-control sayaç kategorisinden, ilk baktığı şey o.

Limit koymak yetmiyor; birinin onu uygulaması gerekiyor. Taradığım 24 harness'ın 12'sinde mekanizma **var ama varsayılanda kapalı**.

08 İZ VE KANIT

~1 dk

```
cua ▶ --trace
cua ▶ Formu doldur ve gönder
# ayrı terminalde:
python3 -m cua_lab.cli replay izler/openhands-stuck.jsonl
```

# eylem	ekran	karar	sebep
0 left_click	80f21d78b1a6	continue	
1 left_click	80f21d78b1a6	continue	
!! 2 left_click	80f21d78b1a6	stop	cycle_k2

SÖYLE

"Ekran hash'i **üç adım boyunca hiç değişmiyor** — döngünün kanıtı burada, ve dedektörün 'nereden anladığı' sorusunun cevabı bu.

Her iterasyon **ayrı bir span**. Arize'in gerekliliği: *'bütün koşum tek bir span'sa 4–19. adımların aynı çağrı olduğunu göremezsin.'* Loop detection her şeyden önce bir **gözlemlenebilirlik** gereksinimi — entegrasyon listenin birinci maddesi bu."

09 İSTEĞE BAĞLI: GERÇEK MODEL

internet gerekir

```
cua ▶ --model → 2 (hf)
cua ▶ --budget max_seconds ▶ 60
cua ▶ --strategy → 5 (openhands-stuck)
cua ▶ Gönder butonuna bas
```

```
OLCULEN (Llama-3.1-8B, healthy senaryosu):
none          40 adım 17623 tok (kestim, 300'e gidiyordu)
openhands-stuck 3 adım  987 tok
arize-control   9 adım 2924 tok  sebep: budget_SECONDS
```

SÖYLE

"Bu sentetik değil. **Gerçek bir model, hiçbir şey bozuk değilken kendiliğinden döngüye girdi** — 40 çağrının 40'ı left_click, 34'ü birebir aynı argümanla.

Ve beklemediğim bir şey oldu: arize-control **süre ekseninden** durdu, adımdan değil. Betikli modellerde koşumlar 0,01 saniye sürüyordu, süre eksenini hiç bağlamıyordu. **Gerçek çıkarım yavaş olunca bağlayan eksen değişti.**

Çok eksenli bütçeyi şimdiye kadar kaynaklara dayanarak savunuyordum; artık kendi ölçümüm var."

△ **hf ile none seçme** — bütçe kancası yok, 300 gerçek API çağrısına gider.

10 DEMO AKSARSA — KURTARMA PLANI

OLURSA	NE YAP / NE SÖYLE
Komut bulunamadı	cd yanlış klasörde. Paket cua_lab/ içinden import ediyor.
Bilinmeyen strateji hatası	Belgede olup koddan olmayan bir isim yazdın — CLI zaten temiz mesaj basıyor ve mevcut listeyi veriyor. Bu bir demo fırsatı: "17 kaynağı belgeledim, 16'sı koddan; çalıştığını gösteremediğimi çalışıyor diye sunmuyorum."
hf takılıyor / internet yok	Ctrl-C, --model → stubborn . Sayıları slaytan oku: 40 adım → 3 adım.
Tablo terminale sığmıyor	Terminali genişlet ya da --compare yerine tek tek run .
Beklenmedik sonuç çıktı	Gizleme, oku. "Bu benim beklediğim değil — nedenini birlikte bakalım" demek, çalışmanın yöntemine uygun. İzi replay ile aç.

11 BEKLENEN SORULAR — HAZIR CEVAPLAR

"Ortam sentetik, gerçek bir masaüstü değil."

Doğru. Ölçtüğüm şey ortamın gerçekliği değil, **kontrolün devreye girip girmediği**. Sentetik ortam **deterministik** — kontrol testinin (16 sütun birebir aynı) mümkün olmasının sebebi bu; gerçek masaüstünde saat ve imleç her karede hash'i değiştirirdi. Modelin gerçekten döngüye girdiğini hf ile ayrıca gösterdim.

"Eşikleri nereden aldın?"

Kaynaklardan — OpenHands 4/3/3/6, OpenClaw 10/20/30. **Tahmin olduğunu kabul ediyorum**, ve 16 numaralı zihniyet (improvement-loop) tam olarak bunun eleştirisi. Bir kaynağın sayısını **ölçerek düzelttiğim** bir vaka da var: loopguard 'ın eşikleri meşru koşumu başarıdan bir adım önce kesiyordu.

Kapanış — tek cümle

Model döngüyü kesemez; **kesme kararı için gereken bilgi onun görüş alanında değil**. Kesme yetkisi dışarıda olmalı — ve dışarısı **ne olduğunu da söyleyebilmeli**.

"Her ajan döngüsünün **tasarlanmış bir çıkışı** olmalıdır — başarı, geri dönüş, sorma veya devretme. 'Sonsuza kadar dene' değil. 'Umarım model çözer' değil." — Modexa

"Hepsini uygulamak gerekiyor mu?"

Hayır. **Öncelik 1** tek başına vakaların çoğunu durduruyor — ama *neden* durduğunu söylemiyor. Önerim: **1 → 16 → 5**. Önce sert tavan, sonra izleri toplayıp eşikleri ölç, sonra dedektörleri **ölçülmüş eşiklerle** yaz.

"Yanlış pozitif riski?"

Ölçtüğüm 108 kombinasyonda sıfır — ama bu iki senaryo, gerçek çeşitlilik değil. Literatürden: deterministik kontroller **0/63**, istatistiksel yaklaşımlar **%17**. Deterministik kuralları tercih etme gerekçem bu.

"Ne kadar iş?"

Öncelik 1 bir öğleden sonra. En büyük kalem **gözlemlenebilirlik** (iterasyon başına span) — mevcut loglama şemasına dokunuyor.